

# IBM Applied Data Science Capstone

## SpaceX Falcon 9 First Stage Landing Prediction

Data Science Capstone Project Report  
Prepared by: Dalal F. S. A. Alhabad

GitHub Repository: <https://github.com/dalalalhabad/IBM-Data-Science-Capstone-SpaceX>

# Submission Evidence and Repository

This slide is included because the IBM rubric explicitly asks for the GitHub URL.

**PDF**

submission file

**7**

completed notebooks

**1**

Dash app file

**CSV**

launch data

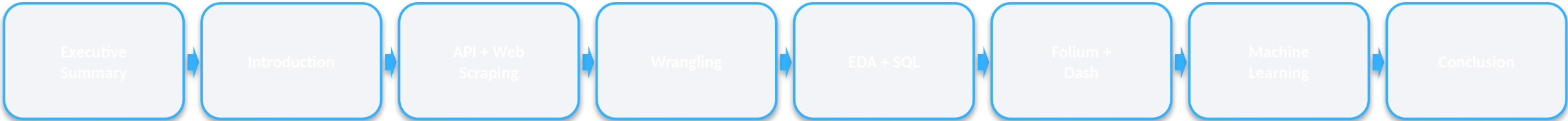
## GitHub repository URL

<https://github.com/dalalalhabad/IBM-Data-Science-Capstone-SpaceX>

- Repository should contain all completed notebooks, the Dash app, CSV files and the final PDF report.
- Do not submit with a placeholder. Create this repository or replace this URL with your actual public project repository before submission.

# Report Roadmap

Built directly around the IBM Applied Data Science Capstone grading criteria



| Rubric requirement                    | Where it appears in this report |
|---------------------------------------|---------------------------------|
| 1.3 Executive Summary                 | Slide 3                         |
| 1.4 Introduction                      | Slide 4                         |
| 1.5-1.7 Data collection and wrangling | Slides 6-11                     |
| 1.8, 1.11 EDA visualisation           | Slides 12-20                    |
| 1.9, 1.12 SQL analysis                | Slides 21-23                    |
| 1.10, 1.13-1.14 Folium and Dash       | Slides 24-29                    |
| 1.15 Predictive analysis              | Slides 30-34                    |

# Executive Summary

90

Falcon 9 launches

66.7%

overall success

83

ML features

83.3%

test accuracy

4

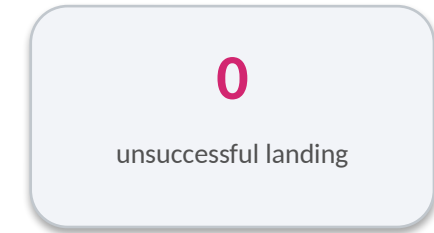
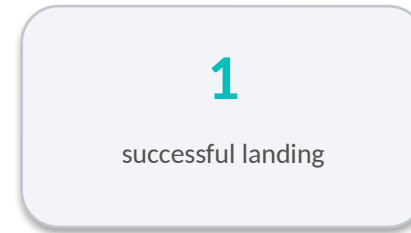
ML models

- Objective: predict whether a Falcon 9 first stage will land successfully after launch.
- Methods: SpaceX API-style collection, Wikipedia web scraping, data wrangling, EDA, SQL, Folium, Dash and supervised machine learning.
- Key result: Logistic Regression, SVM, Decision Tree and KNN all achieved 83.3% test accuracy on the held-out test set.
- Final model selected: Logistic Regression, because it reached the same test accuracy while remaining simple and interpretable.

# Introduction and Business Problem

Why predicting first-stage recovery matters

- SpaceX Falcon 9 missions are designed to recover and reuse the first-stage booster.
- Successful recovery can lower launch cost and improve mission planning.
- The target variable is binary: Class = 1 means successful landing; Class = 0 means failed or no landing attempt.
- The modelling problem is to estimate landing success using launch site, payload, orbit, booster and flight information.



# End-to-End Data Science Workflow

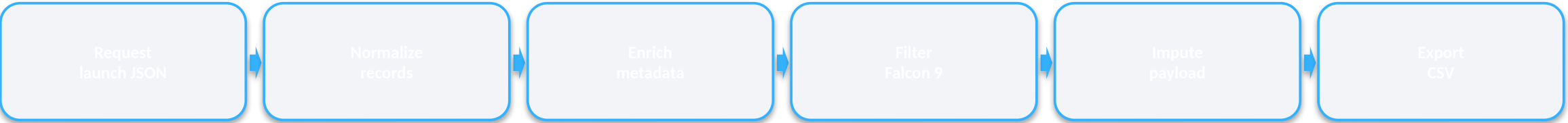
From raw records to deployed dashboard and predictive model



| Stage             | Output file / result              | Purpose                          |
|-------------------|-----------------------------------|----------------------------------|
| API collection    | dataset_part_1.csv                | raw Falcon 9 launch table        |
| Wrangling         | dataset_part_2.csv                | Class target and cleaned records |
| EDA preprocessing | dataset_part_3.csv                | one-hot encoded features         |
| SQL               | SPACEXTABLE                       | query-based analysis             |
| Folium            | interactive HTML map              | launch-site geospatial analysis  |
| Dash              | spacex_dash_app_FIXED.py          | interactive visual analytics     |
| ML                | model scores + confusion matrices | landing prediction               |

# Data Collection - SpaceX API Methodology

API calls and reproducible extraction workflow



- The notebook requested SpaceX launch records, normalized JSON into tabular form and prepared an enriched Falcon 9 dataset.
- Because some live SpaceX endpoints returned invalid JSON during execution, the official IBM static dataset was used as the reproducible source.
- Final API-style dataset: 90 Falcon 9 records and 17 columns.

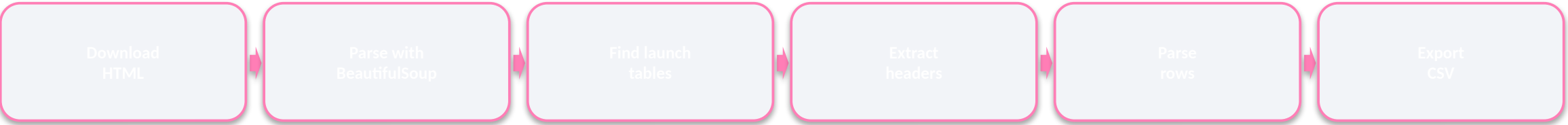
| Output                               | Value       |
|--------------------------------------|-------------|
| Raw static launch records            | 107 x 42    |
| Final Falcon 9 records               | 90 x 17     |
| Mean payload used for imputation     | 6,104.96 kg |
| Missing PayloadMass after imputation | 0           |





# Data Collection - Web Scraping Methodology

Extracting launch records from Wikipedia tables



- BeautifulSoup parsed the launch-record table, removed footnotes and cleaned text fields.
- The notebook extracted flight number, date, booster version, launch site, payload, payload mass, orbit, customer, launch outcome and booster landing status.
- Validation confirmed all parsed columns had the same length.

| Web scraping result    | Value    |
|------------------------|----------|
| HTML tables found      | 25       |
| Launch rows extracted  | 124      |
| Malformed rows skipped | 0        |
| Final scraped dataset  | 124 x 11 |
| Missing payload mass   | 2        |

# Data Collection - Web Scrapping Output Evidence

Notebook output with 124 extracted rows and dataframe preview

05\_SpaceX\_EDA\_SQL\_FIXED\_FINAL.ipynb06\_SpaceX\_Folium\_Launch\_Site\_Analysis\_FIXED\_FINAL.ipynbSpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb01\_SpaceX\_Data\_Collection\_API\_FIXED\_FINAL.ipynb02\_SpaceX\_Web\_Scraping\_FIXED\_FINAL.ipynb03\_SpaceX\_Data\_Wrangling\_FIXED\_FINAL.ipynb04Python 3

Users > dalalalhabad > Desktop > 02\_SpaceX\_Web\_Scraping\_FIXED\_FINAL.ipynb > H1 empty cell

GenerateCodeMarkdownRun AllRestartClear All OutputsJupyter VariablesOutline

for key, values in launch\_dict.items():

print("Column lengths:", column\_lengths)

if extracted\_row == 0:

raise ValueError("No launch records were extracted.")

if len(set(column\_lengths.values())) != 1:

raise ValueError("The parsed columns do not all have the same length.")

(13)

Launch rows extracted: 124

Malformed rows skipped: 0

Column lengths: {'Flight No.': 124, 'Date': 124, 'Time': 124, 'Version Booster': 124, 'Launch site': 124, 'Payload': 124, 'Payload mass': 124, 'Orbit': 124, 'Customer': 124, 'Launch outcome': 124, 'Booster landing': 124}

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

# Create the final Pandas DataFrame.

df = pd.DataFrame(launch\_dict)

# Convert Flight No. to a nullable numeric type where possible,

df["Flight No."] = pd.to\_numeric(df["Flight No."], errors="coerce").astype("Int64")

# Remove accidental duplicate launch rows while preserving order.

df = df.drop\_duplicates().reset\_index(drop=True)

print(f"Final web-scraped dataset: {df.shape[0]} rows and {df.shape[1]} columns")

df.head(10)

(14)

Final web-scraped dataset: 124 rows and 11 columns

|   | Flight No. | Date              | Time  | Version         | Booster                | Launch site                          | Payload  | Payload mass | Orbit           | Customer          | Launch outcome | Booster landing      |
|---|------------|-------------------|-------|-----------------|------------------------|--------------------------------------|----------|--------------|-----------------|-------------------|----------------|----------------------|
| 0 | 1          | 4 June 2010       | 18:45 | F9 v1.0 B0003.1 | CCAFS , SLG-40         | Dragon Spacecraft Qualification Unit |          | NaN          | LEO             | SpaceX            | Success        | Failure (parachute)  |
| 1 | 2          | 8 December 2010   | 15:43 | F9 v1.0 B0004.1 | CCAFS , SLG-40         | Dragon demo flight C1 (Dragon C101)  |          | NaN          | LEO ( ISS )     | NASA ( COTS ) NRO | Success        | Failure (parachute)  |
| 2 | 3          | 22 May 2012       | 07:44 | F9 v1.0 B0005.1 | CCAFS , SLG-40         | Dragon demo flight C2+ (Dragon C102) | 525 kg   |              | LEO ( ISS )     | NASA ( COTS )     | Success        | No attempt           |
| 3 | 4          | 8 October 2012    | 00:35 | F9 v1.0 B0006.1 | CCAFS , SLG-40         | SpaceX CRS-1 (Dragon C103)           | 4,700 kg |              | LEO ( ISS )     | NASA ( CRS )      | Success        | No attempt           |
| 4 | 5          | 1 March 2013      | 15:10 | F9 v1.0 B0007.1 | CCAFS , SLG-40         | SpaceX CRS-2 (Dragon C104)           | 4,877 kg |              | LEO ( ISS )     | NASA ( CRS )      | Success        | No attempt           |
| 5 | 6          | 29 September 2013 | 16:00 | F9 v1.1 B1003   | VAFB , SLG-4E          | CASSIOPE                             | 500 kg   |              | Polar orbit LEO | MDA               | Success        | Uncontrolled (ocean) |
| 6 | 7          | 3 December 2013   | 22:41 | F9 v1.1 B1004   | CCAFS , SLG-40         | SES-8                                | 3,170 kg |              | GTO             | SES               | Success        | No attempt           |
| 7 | 8          | 6 January 2014    | 22:06 | F9 v1.1         | CCAFS , SLG-40         | Thalcom 6                            | 3,325 kg |              | GTO             | Thalcom           | Success        | No attempt           |
| 8 | 9          | 18 April 2014     | 19:25 | F9 v1.1         | Cape Canaveral , LC-40 | SpaceX CRS-3 (Dragon C105)           | 2,296 kg |              | LEO ( ISS )     | NASA ( CRS )      | Success        | Controlled (ocean)   |
| 9 | 10         | 14 July 2014      | 15:15 | F9 v1.1         | Cape Canaveral , LC-40 | Orbcomm-OG2 -1 (6 satellites)        | 1,316 kg |              | LEO             | Orbcomm           | Success        | Controlled (ocean)   |

We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.

Following labs will be using a provided dataset to make each lab independent.

# Validate and export the web-scraped dataset.

required\_columns = list(launch\_dict.keys())

missing\_columns = {

column for column in required\_columns

if column not in df.columns

}

if missing\_columns:

raise ValueError(f"Missing required columns: {missing\_columns}")

if df.empty:

(15)

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTSJUPYTERQUERY RESULTS

dalalalhabad@Dalals-MacBook-Air: Desktop % python3 spacex\_dash\_app\_FIXED.py

python3 spacex\_dash\_app\_FIXED.py

Dash is running on http://127.0.0.1:8858/

\* Serving Flask app 'spacex\_dash\_app\_FIXED'

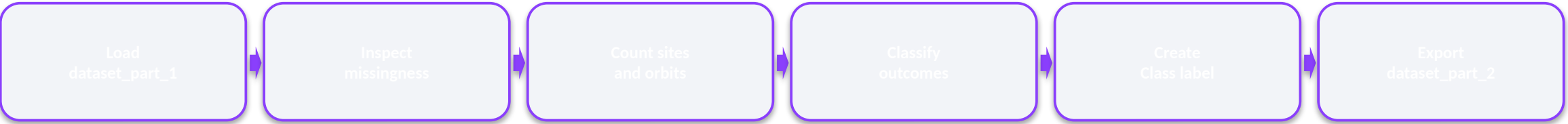
\* Debug mode: on

dalalalhabad@Dalals-MacBook-Air: Desktop %

zshPython

# Data Wrangling Methodology

Cleaning and encoding landing outcomes



- Landing outcome strings were converted to a binary target: 1 = successful landing, 0 = unsuccessful/no landing.
- The final wrangled dataset contained 90 launches, 18 variables and no missing target labels.
- Overall landing success after wrangling was 66.7%.

| Class        | Count | Share |
|--------------|-------|-------|
| Unsuccessful | 30    | 33.3% |
| Successful   | 60    | 66.7% |
| Total        | 90    | 100%  |

# Data Wrangling Output Evidence

## Landing outcome counts and unsuccessful outcome definitions

### TASK 3: Calculate the number and occurrence of mission outcome of the orbits

Use the method `.value_counts()` on the column `Outcome` to determine the number of `landing_outcomes`. Then assign it to a variable `landing_outcomes`.

```
# TASK 3: Calculate the number of occurrences of each landing outcome.
landing_outcomes = df["Outcome"].value_counts(dropna=False)

print("Landing outcome counts:")
landing_outcomes
```

```
Landing outcome counts:
Outcome
True ASDS      41
None None      19
True RTLS      14
False ASDS       6
True Ocean       5
False Ocean      2
None ASDS        2
False RTLS       1
Name: count, dtype: int64
```

`True Ocean` means the mission outcome was successfully landed to a specific region of the ocean while `False Ocean` means the mission outcome was unsuccessfully landed to a specific region of the ocean. `True RTLS` means the mission outcome was successfully landed to a ground pad `False RTLS` means the mission outcome was unsuccessfully landed to a ground pad. `True ASDS` means the mission outcome was successfully landed to a drone ship `False ASDS` means the mission outcome was unsuccessfully landed to a drone ship. `None ASDS` and `None None` these represent a failure to land.

```
# Display each landing outcome with its position in the value-count result.
for index, outcome in enumerate(landing_outcomes.index):
    print(index, outcome)
```

```
0 True ASDS
1 None None
2 True RTLS
3 False ASDS
4 True Ocean
5 False Ocean
6 None ASDS
7 False RTLS
```

We create a set of outcomes where the second stage did not land successfully:

# EDA with Data Visualization - Methodology

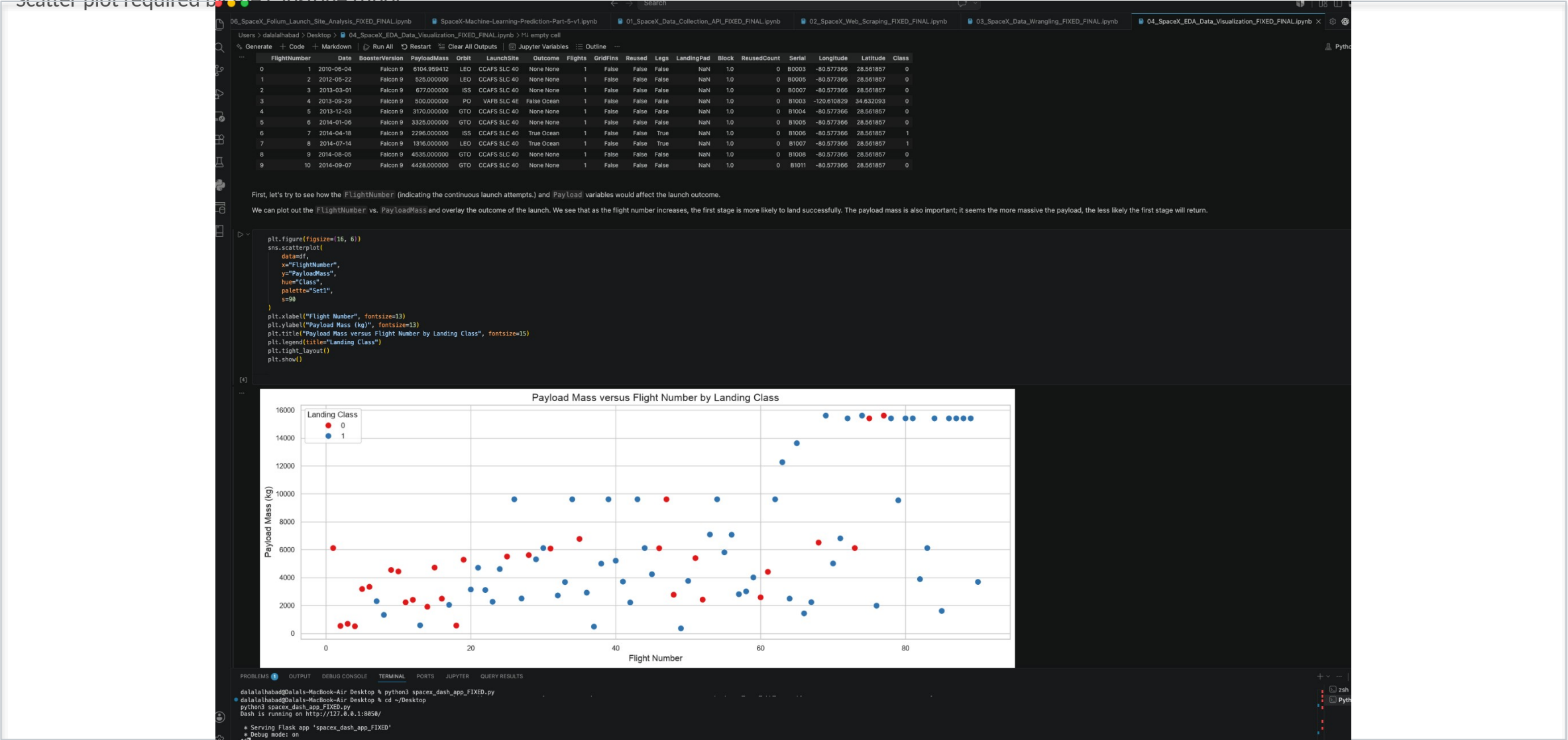
Charts used to understand patterns before modelling

| Chart                         | Purpose                                                            |
|-------------------------------|--------------------------------------------------------------------|
| Payload Mass vs Flight Number | Inspect relationship between payload, experience and landing class |
| Flight Number vs Launch Site  | Identify launch-site trends over time                              |
| Payload Mass vs Launch Site   | Compare payload ranges across sites                                |
| Success Rate by Orbit         | Find orbit categories linked to landing success                    |
| Flight Number vs Orbit        | Assess temporal patterns by orbit                                  |
| Payload Mass vs Orbit         | Relate payload and orbit type                                      |
| Yearly Success Trend          | Show improvement over time                                         |

- These charts provide the evidence required for the EDA visualisation result slides.

# EDA Result - Payload Mass versus Flight Number

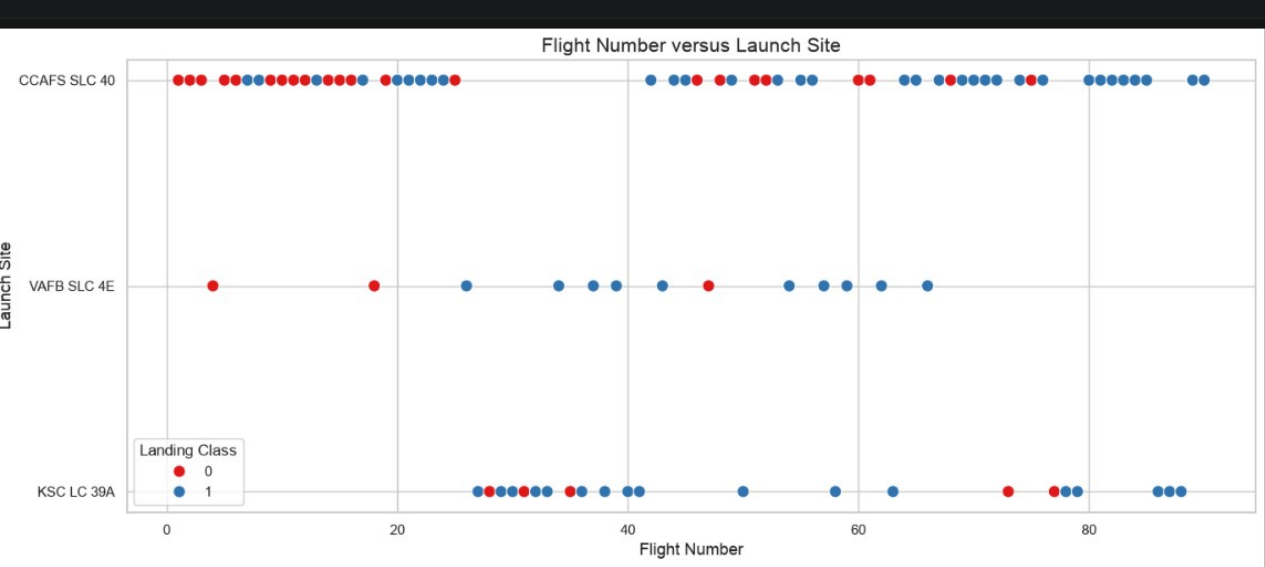
Scatter plot required by the capstone rubric



# EDA Result - Flight Number versus Launch Site

Launch site success pattern with notebook output table

```
site_flight_summary = (
    df.groupby("LaunchSite")
        .agg(
            Launches=("FlightNumber", "count"),
            First_Flight=("FlightNumber", "min"),
            Last_Flight=("FlightNumber", "max"),
            Success_Rate=("Class", "mean")
        )
        .sort_values("Success_Rate", ascending=False)
)
```



|              | Launches | First_Flight | Last_Flight | Success_Rate |
|--------------|----------|--------------|-------------|--------------|
| LaunchSite   |          |              |             |              |
| KSC LC 39A   | 22       | 27           | 88          | 0.772727     |
| VAFB SLC 4E  | 13       | 4            | 66          | 0.769231     |
| CCAFS SLC 40 | 55       | 1            | 90          | 0.600000     |

Now try to explain the patterns you found in the Flight Number vs. Launch Site scatter point plots.

## TASK 2: Visualize the relationship between Payload and Launch Site

We also want to observe if there is any relationship between launch sites and their payload mass.

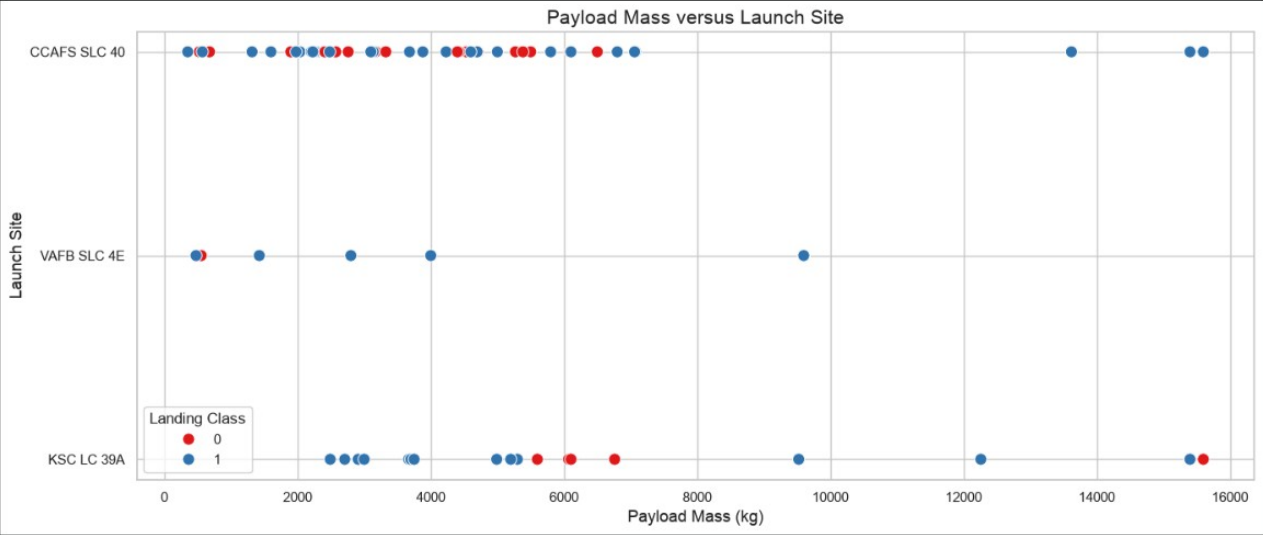
# EDA Result - Payload Mass versus Launch Site

Payload range and site specific success differences

```
plt.legend(title="Landing Class")
plt.tight_layout()
plt.show()

payload_site_summary = (
    df.groupby("LaunchSite")
    .agg(
        Launches=("PayloadMass", "count"),
        Minimum_Payload=("PayloadMass", "min"),
        Maximum_Payload=("PayloadMass", "max"),
        Average_Payload=("PayloadMass", "mean"),
        Success_Rates=("Class", "mean")
    )
    .round(2)
)

payload_site_summary
```



|              | Launches | Minimum_Payload | Maximum_Payload | Average_Payload | Success_Rate |
|--------------|----------|-----------------|-----------------|-----------------|--------------|
| LaunchSite   |          |                 |                 |                 |              |
| CCAFS SLC 40 | 55       | 350.0           | 15600.0         | 5548.21         | 0.60         |
| KSC LC 39A   | 22       | 2490.0          | 15600.0         | 7606.45         | 0.77         |
| VAFB SLC 4E  | 13       | 475.0           | 9600.0          | 5919.46         | 0.77         |

Now if you observe Payload Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).

TASK 3: Visualize the relationship between success rate of each orbit type

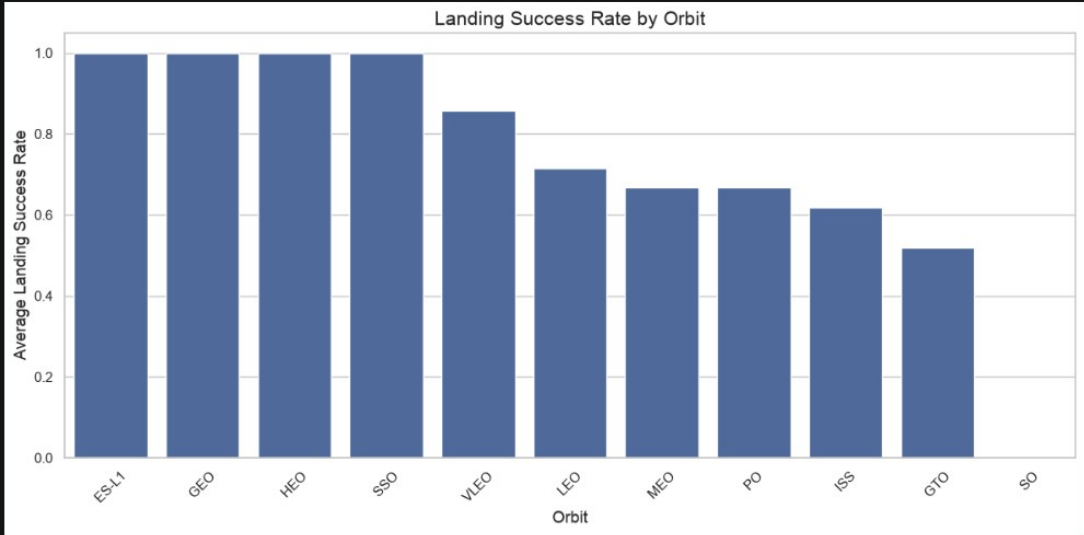


# EDA Result - Landing Success Rate by Orbit

Bar chart and table output from the EDA notebook

```
sns.barplot(
    data=orbit_success_rate,
    x="Orbit",
    y="SuccessRate"
)
plt.xlabel("Orbit", fontsize=13)
plt.ylabel("Average Landing Success Rate", fontsize=13)
plt.title("Landing Success Rate by Orbit", fontsize=15)
plt.ylim(0, 1.05)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

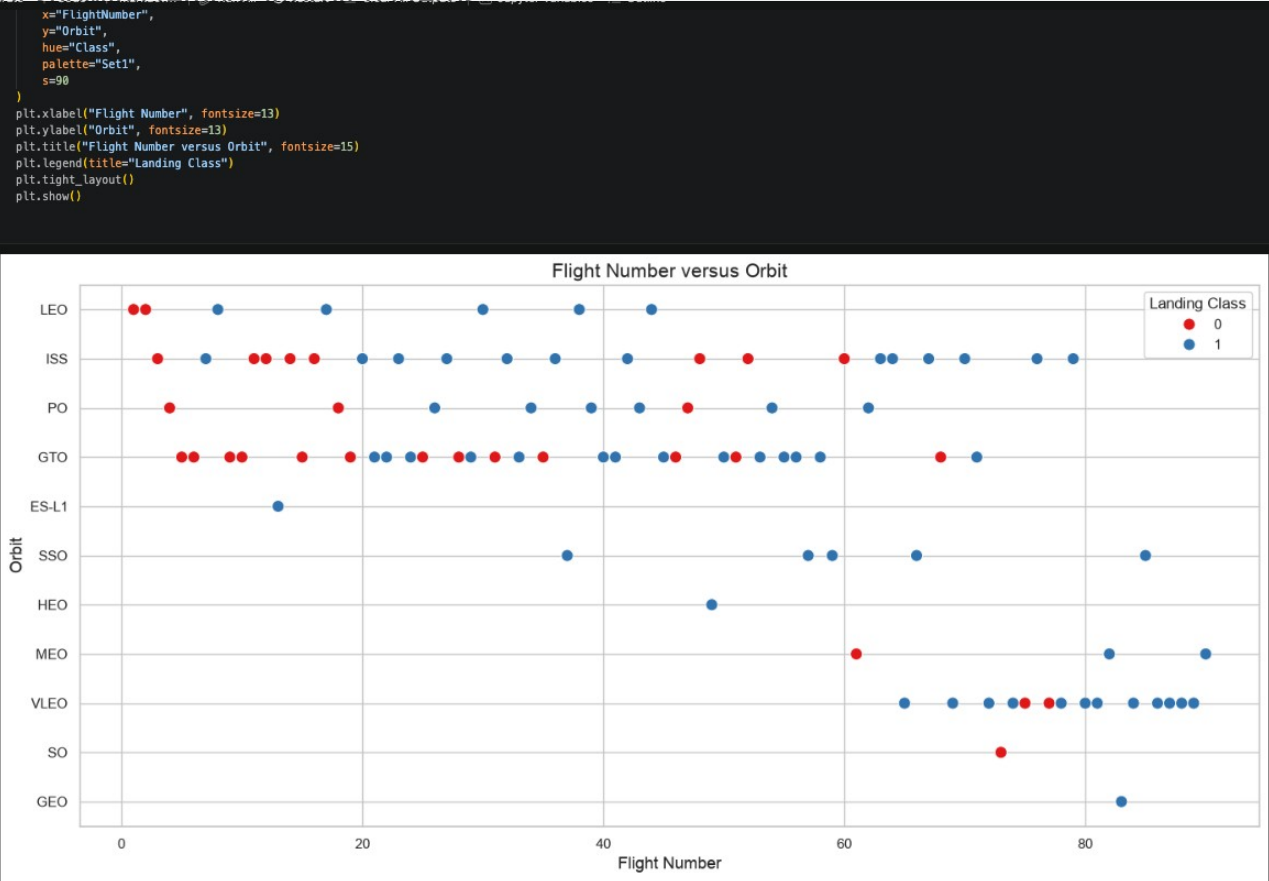
orbit_success_rate
```



|    | Orbit | SuccessRate |
|----|-------|-------------|
| 0  | ES-L1 | 1.000000    |
| 1  | GEO   | 1.000000    |
| 3  | HEO   | 1.000000    |
| 9  | SSO   | 1.000000    |
| 10 | VLEO  | 0.857143    |
| 5  | LEO   | 0.714286    |
| 6  | MEO   | 0.666667    |
| 7  | PO    | 0.666667    |
| 4  | ISS   | 0.619048    |
| 2  | GTO   | 0.518519    |
| 8  | SO    | 0.000000    |

# EDA Result - Flight Number versus Orbit

Scatter plot showing temporal/orbit patterns



You should see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

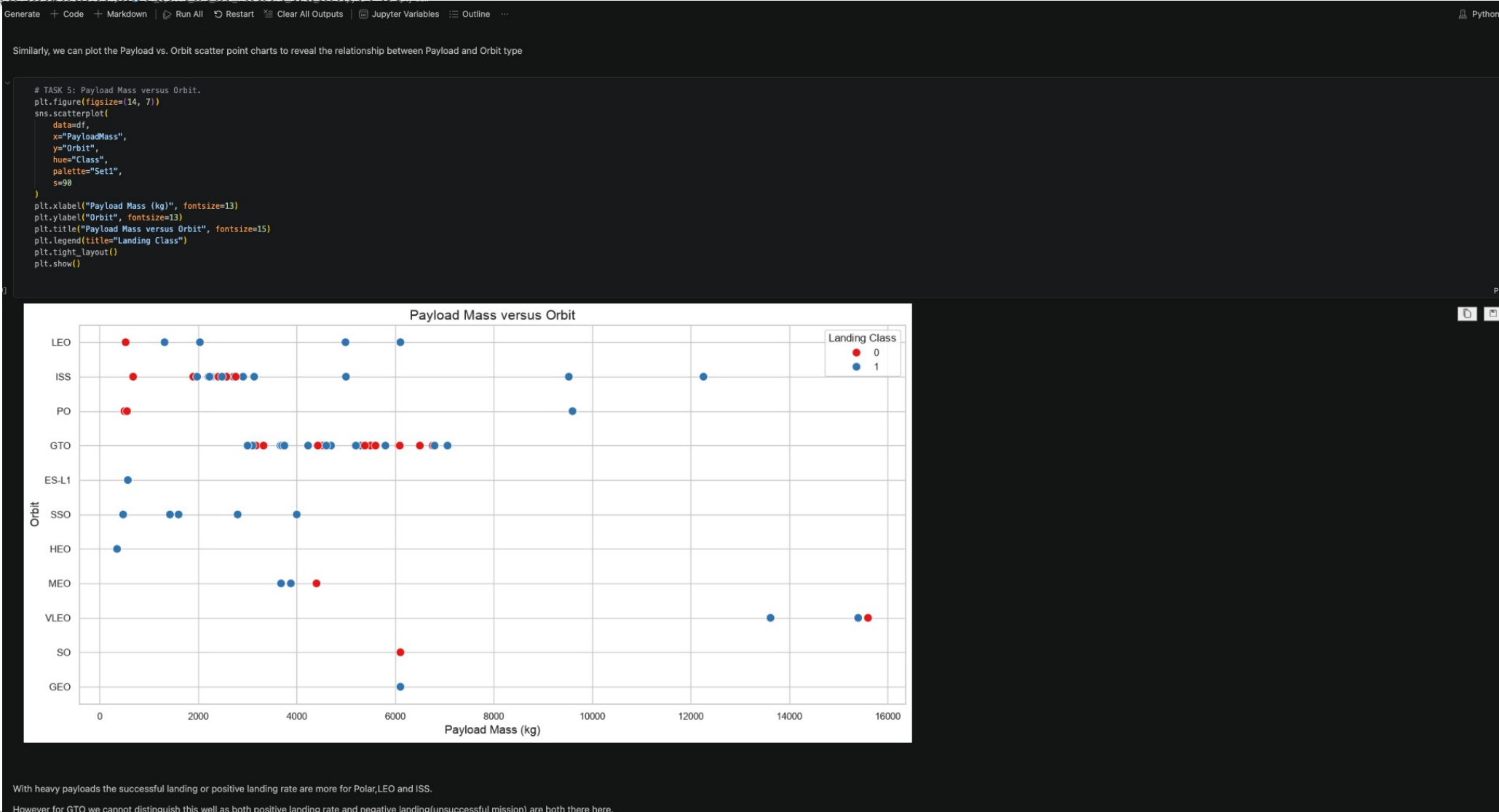
## TASK 5: Visualize the relationship between Payload and Orbit type

Similarly, we can plot the Payload vs. Orbit scatter point charts to reveal the relationship between Payload and Orbit type

```
# TASK 5: Payload Mass versus Orbit.
```

# EDA Result - Payload Mass versus Orbit

Scatter plot linking payload, orbit and landing class



# EDA Result - Yearly Landing Success Trend

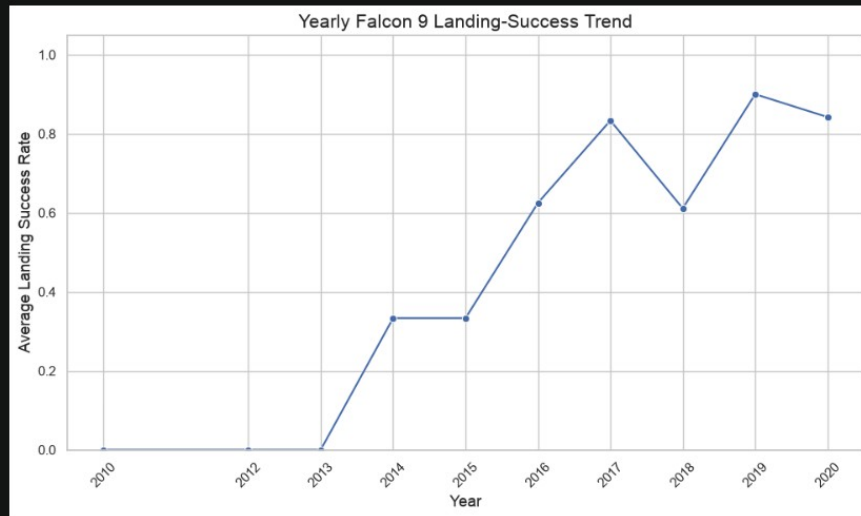
Landing success improved over time

```
yearly_success["Year"] = yearly_success["Year"].astype(int)

plt.figure(figsize=(10, 6))
sns.lineplot(
    data=yearly_success,
    x="Year",
    y="SuccessRate",
    marker="o"
)

plt.xlabel("Year", fontsize=13)
plt.ylabel("Average Landing Success Rate", fontsize=13)
plt.title("Yearly Falcon 9 Landing-Success Trend", fontsize=15)
plt.ylim(0, 1.05)
plt.xticks(yearly_success["Year"].unique(), rotation=45)
plt.tight_layout()
plt.show()

yearly_success
```



|   | Year | SuccessRate |
|---|------|-------------|
| 0 | 2010 | 0.000000    |
| 1 | 2012 | 0.000000    |
| 2 | 2013 | 0.000000    |
| 3 | 2014 | 0.333333    |
| 4 | 2015 | 0.333333    |
| 5 | 2016 | 0.625000    |
| 6 | 2017 | 0.833333    |
| 7 | 2018 | 0.611111    |
| 8 | 2019 | 0.900000    |
| 9 | 2020 | 0.842105    |

# EDA Output - Feature Engineering for Machine Learning

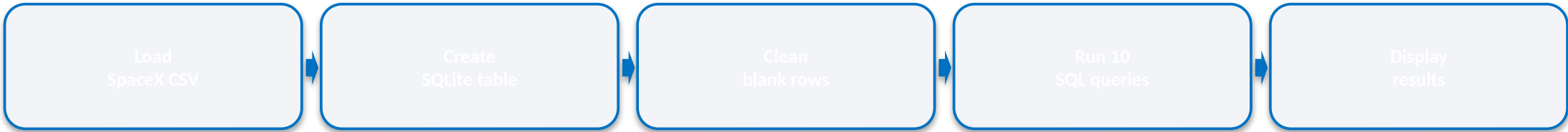
One-hot encoding prepared categorical variables for supervised learning



- Categorical variables such as Orbit, LaunchSite, LandingPad and Serial were converted into dummy variables.
- All final features were cast to float64 so that scikit-learn models could train consistently.
- Output file: dataset\_part\_3.csv.

# EDA with SQL - Methodology

SQLite queries used to answer required capstone questions



- The SQL notebook used a SQLite table named SPACEXTABLE.
- Queries covered launch sites, payload totals, booster versions, successful ground pad landings, mission outcomes, maximum payloads and time-based failure analysis.

| SQL task type    | Example output                  |
|------------------|---------------------------------|
| Launch sites     | Four unique launch-site records |
| Payload totals   | NASA CRS payload = 45,596 kg    |
| Landing outcomes | Success = 100, Failure = 1      |
| Time analysis    | 2015 failed drone-ship landings |

# EDA with SQL Results - Launch Sites and Payloads

Required SQL outputs covering sites, payloads and rankings

| Question                            | SQL result                                         |
|-------------------------------------|----------------------------------------------------|
| Unique launch sites                 | CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E |
| NASA (CRS) total payload            | 45,596 kg                                          |
| Average payload - F9 v1.1           | 2,928.4 kg                                         |
| First successful ground pad landing | 2015-12-22                                         |
| Max payload boosters                | B1048, B1049, B1051, B1056, B1058, B1060           |

- These results satisfy the rubric requirement for launch sites, payload analysis and ranking outputs.

# EDA with SQL Results - Mission Outcomes and Time Analysis

Required SQL outputs covering success counts and date-based failures

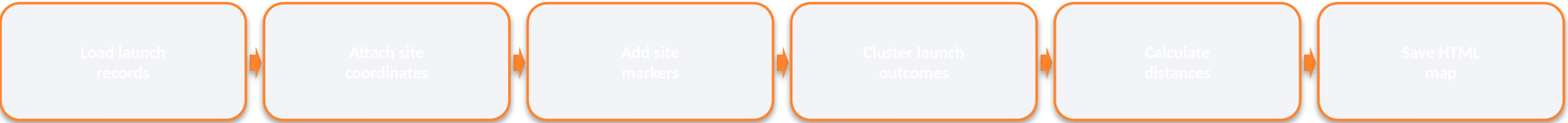
| SQL query theme                                                | Result                                      |
|----------------------------------------------------------------|---------------------------------------------|
| Mission outcomes                                               | Success = 100; Failure = 1                  |
| Failed drone ship landings in 2015                             | January and April 2015 failure records      |
| Landing outcome ranking 2010-06-04 to 2017-03-20               | Outcome counts ordered by frequency         |
| Boosters with success on drone ship and payload 4,000-6,000 kg | Returned booster-version set from SQL query |

- The SQL section includes the query goals and result tables rather than only narrative summaries.
- These outputs directly support IBM rubric item 1.12.



# Interactive Visual Analytics - Folium Methodology

Launch-site markers, outcome clusters and proximity analysis

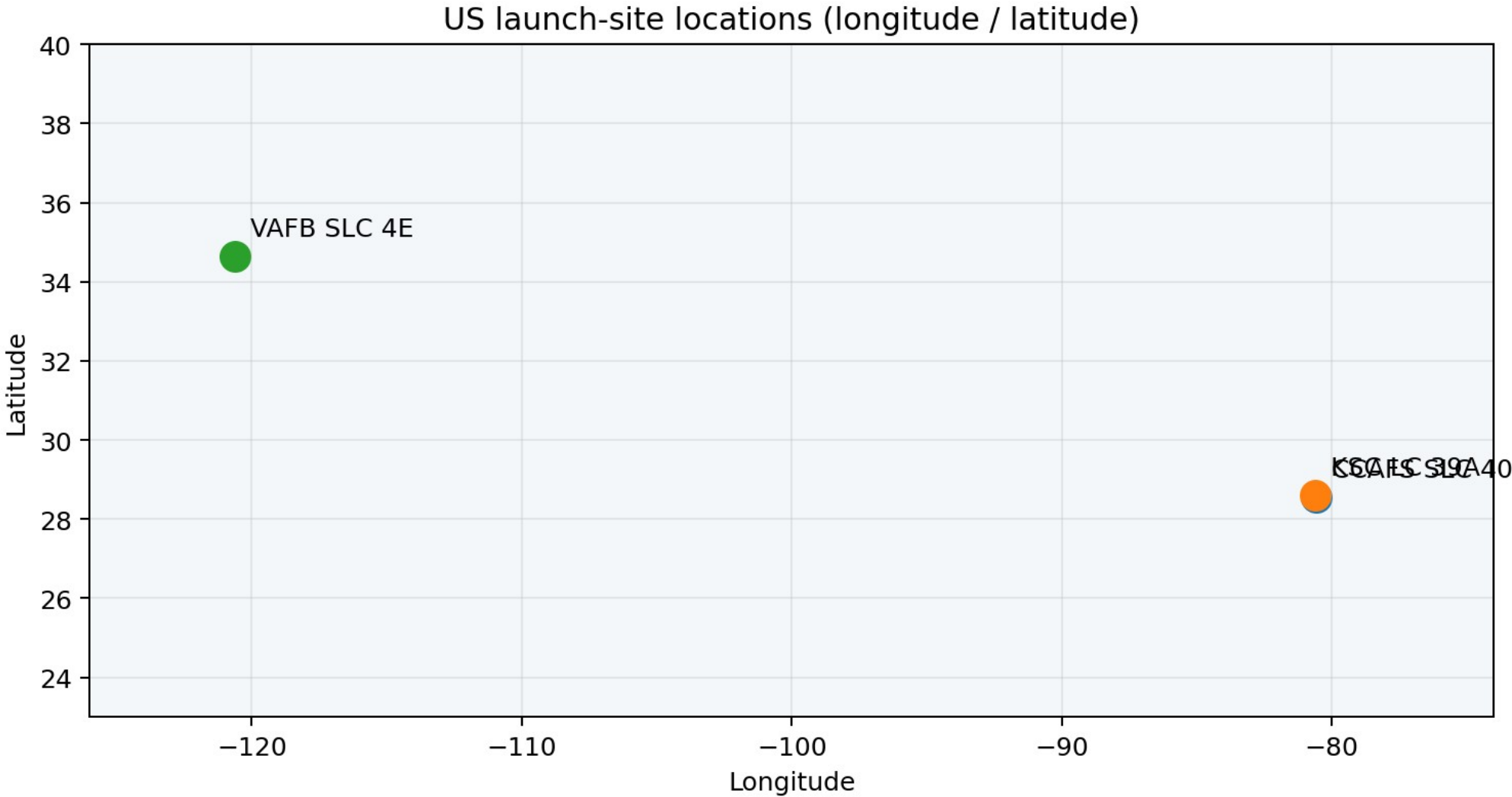


- Folium was used to build an interactive map of launch sites.
- Markers represented launch sites and individual successful/failed launch records.
- Distance calculations measured proximity to coastline, city, highway and railway points.

| Map output                    | Value   |
|-------------------------------|---------|
| Launch sites                  | 3       |
| Successful markers            | 60      |
| Failed markers                | 30      |
| KSC to selected coastline     | 6.54 km |
| KSC to selected railway point | 0.77 km |

# Folium Map Results - Launch Site Markers

Notebook screenshot showing interactive map output

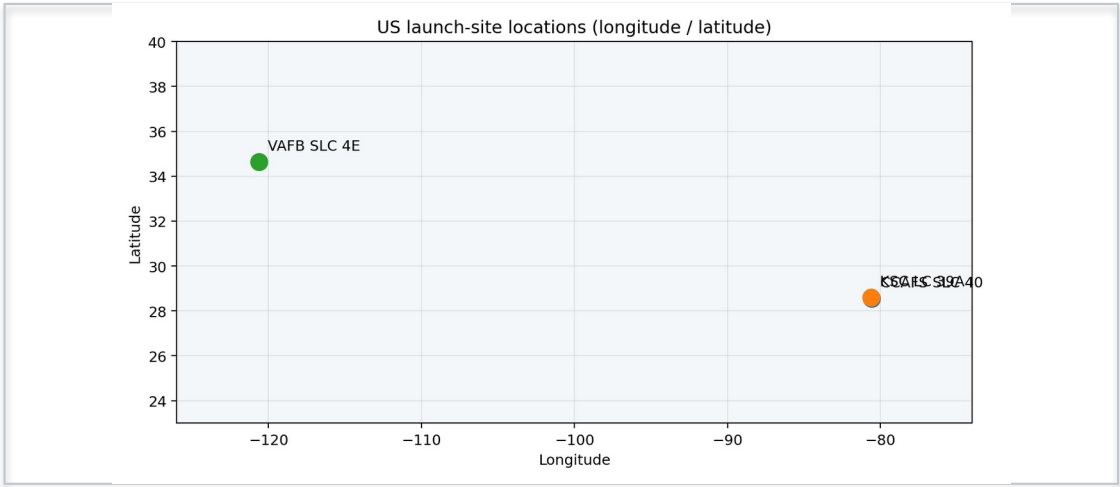


# Folium Proximity Analysis

Distances from KSC LC-39A to selected nearby features

| Feature                  | Approx. distance from KSC LC-39A |
|--------------------------|----------------------------------|
| Selected coastline point | 6.54 km                          |
| Selected city area       | 19.57 km                         |
| Selected highway point   | 3.38 km                          |
| Selected railway point   | 0.77 km                          |

- Launch sites are positioned close to the coastline, which supports safe launch trajectories and booster landing logistics.
- The distance analysis provides a geospatial explanation of launch-site surroundings, satisfying the Folium proximity requirement.



# Interactive Visual Analytics - Plotly Dash Methodology

Dashboard components and callback logic

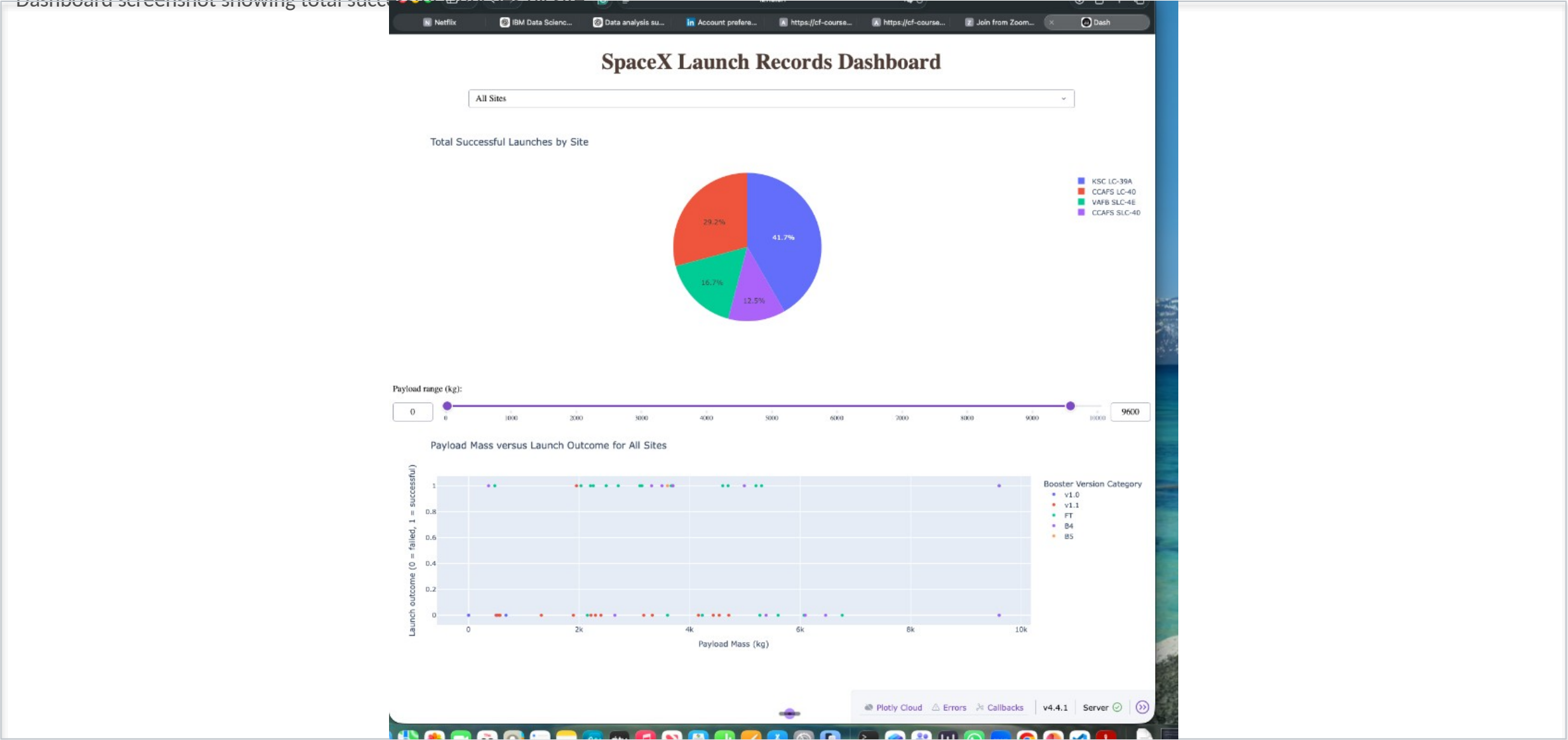
| Dash component                         | Role                                                                |
|----------------------------------------|---------------------------------------------------------------------|
| Dropdown: site-dropdown                | Select all sites or a specific launch site                          |
| Pie chart: success-pie-chart           | Display successful launches by site or success/failure for one site |
| RangeSlider: payload-slider            | Filter payload mass range from 0 to 10,000 kg                       |
| Scatter: success-payload-scatter-chart | Show payload mass versus launch outcome by booster category         |

- The Dash app loads `spacex_launch_dash.csv` and uses callbacks to update the pie chart and scatter plot when the dropdown or slider changes.
- The submitted app code contains all four required tasks: dropdown, pie callback, payload slider and scatter callback.

Code evidence: `spacex_dash_app_FIXED.py` includes `dcc.Dropdown`, `dcc.RangeSlider` and two callback functions.

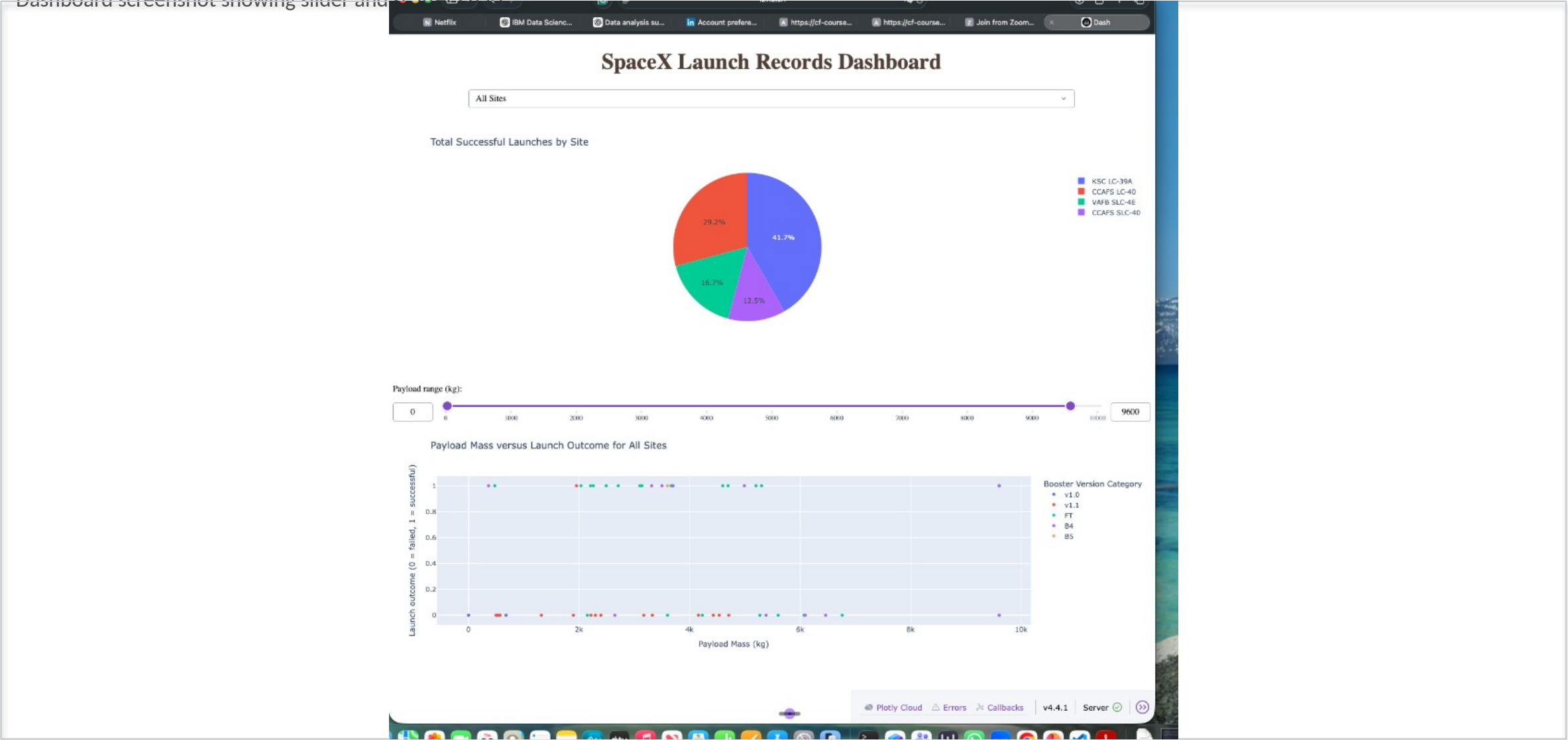
# Plotly Dash Results - Success Pie Chart

Dashboard screenshot showing total successful launches by site



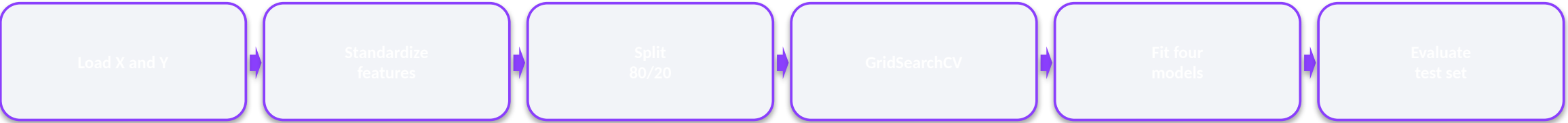
# Plotly Dash Results - Payload versus Launch Outcome

Dashboard screenshot showing slider and scatter chart



# Predictive Analysis - Classification Methodology

Train/test split, scaling and GridSearchCV hyperparameter tuning



| Dataset item | Value       |
|--------------|-------------|
| Samples      | 90          |
| Features     | 83          |
| Training set | 72 launches |
| Test set     | 18 launches |
| Target       | Class (0/1) |

| Model               | Hyperparameter search            |
|---------------------|----------------------------------|
| Logistic Regression | C, penalty, solver               |
| SVM                 | kernel, C, gamma                 |
| Decision Tree       | criterion, depth, split settings |
| KNN                 | n_neighbors, algorithm, p        |

# Predictive Analysis Results - Model Comparison

All four supervised models reached the same held-out test accuracy

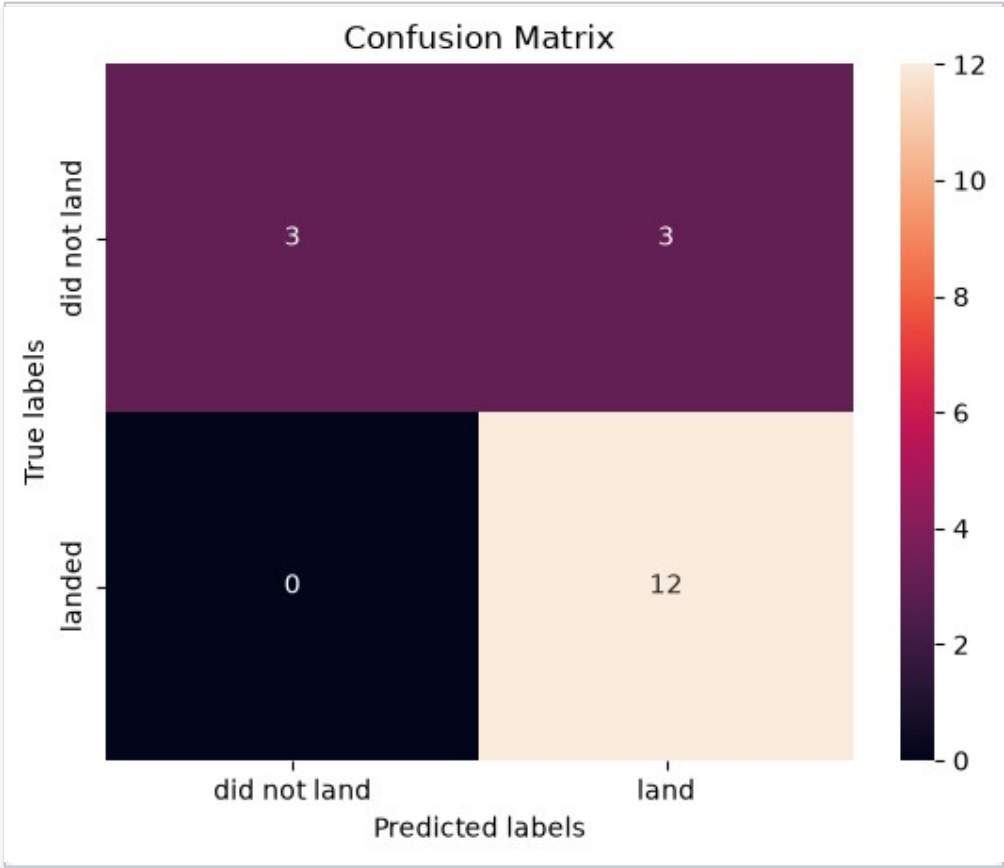
| Model               | Best parameters            | CV accuracy | Test accuracy |
|---------------------|----------------------------|-------------|---------------|
| Logistic Regression | C=0.01, L2, lbfgs          | 84.64%      | 83.33%        |
| SVM                 | C=1, gamma=0.0316, sigmoid | 84.82%      | 83.33%        |
| Decision Tree       | entropy, depth=2, sqrt     | 86.07%      | 83.33%        |
| KNN                 | n=10, p=1                  | 84.82%      | 83.33%        |

- Decision Tree achieved the highest cross-validation accuracy, but all models tied on the held-out test set.
- Logistic Regression was selected as the final model because it is simple, interpretable and matched the best test accuracy.



# Predictive Analysis Results - Confusion Matrix

Logistic Regression achieved 83.33% test accuracy

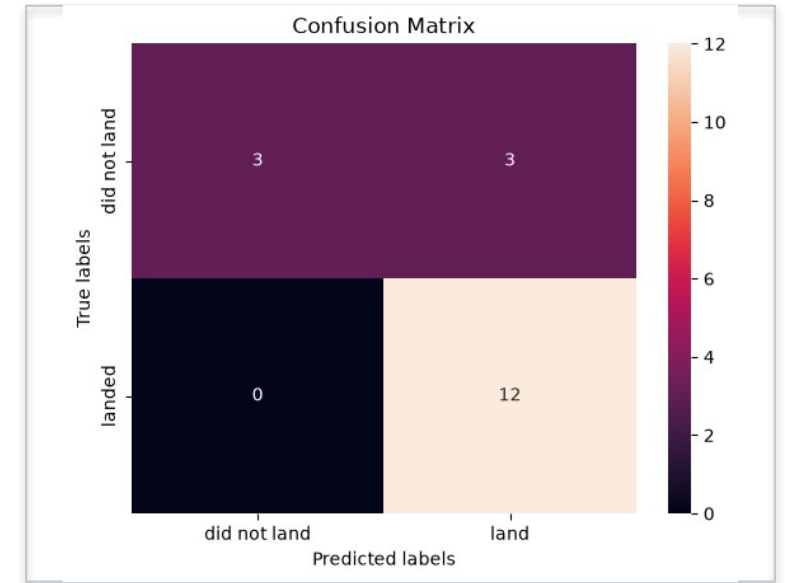
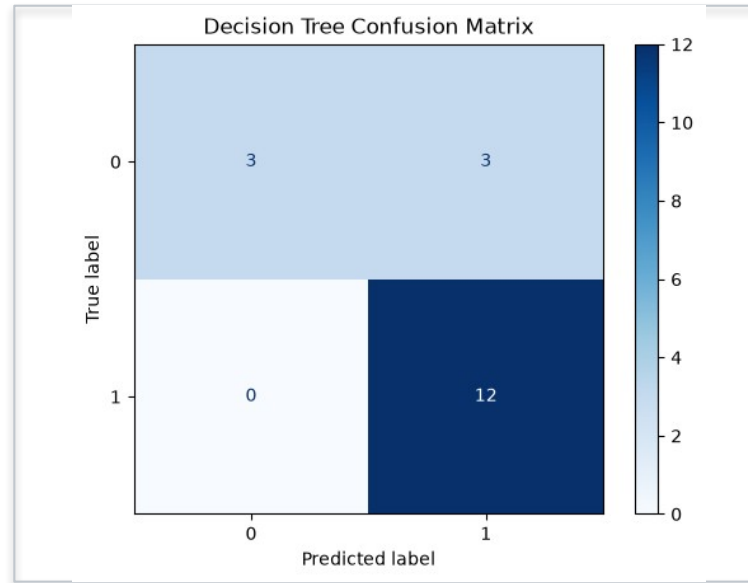
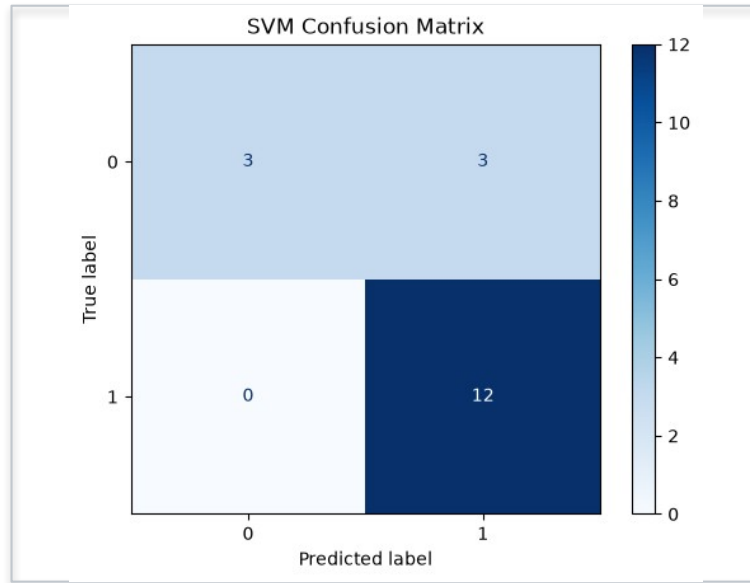


| Metric          | Value  |
|-----------------|--------|
| Test samples    | 18     |
| True negatives  | 3      |
| False positives | 3      |
| False negatives | 0      |
| True positives  | 12     |
| Test accuracy   | 83.33% |

- The model correctly classified all successful landings in the test set.
- The main error pattern was predicting landing success for three cases that did not land.

# Predictive Analysis Results - Model Diagnostics

Confusion matrices for SVM, Decision Tree and KNN



All alternative models achieved the same 83.33% test accuracy as Logistic Regression. The similar confusion matrices indicate that the test set was small and the models learned similar decision boundaries.

# Key Findings and Innovative Insights

What the end-to-end analysis revealed

- Launch site matters: KSC LC 39A and VAFB SLC 4E showed higher landing success rates than CCAFS SLC 40.
- Mission profile matters: orbit type was strongly associated with landing outcome; GTO missions had a lower success rate than several other orbit categories.
- Operational learning is visible: landing success improved substantially after 2015, showing the value of experience and booster reuse improvements.
- Predictive value: a simple supervised learning model predicted landing outcomes with 83.33% test accuracy.
- Geospatial insight: launch facilities are positioned close to coastline and transport infrastructure, supporting safety and logistics requirements.

# Conclusion

End-to-end data science workflow and business value

- This project completed the full Applied Data Science Capstone workflow: data collection, scraping, cleaning, EDA, SQL, mapping, dashboarding and classification.
- The final model provides an interpretable prediction of first-stage landing success using historical launch attributes.
- Interactive Folium and Dash outputs support stakeholder exploration of launch sites, payloads and booster-version patterns.
- Future work could add weather, mission cost, booster age, landing-zone conditions and additional recent launches to improve predictive reliability.

| Deliverable                  | Status                                    |
|------------------------------|-------------------------------------------|
| Final PDF report             | Ready to submit after GitHub URL is valid |
| All notebooks completed      | Yes                                       |
| Dash app completed           | Yes                                       |
| Figures and outputs included | Yes                                       |
| GitHub repository            | Create/update before submission           |

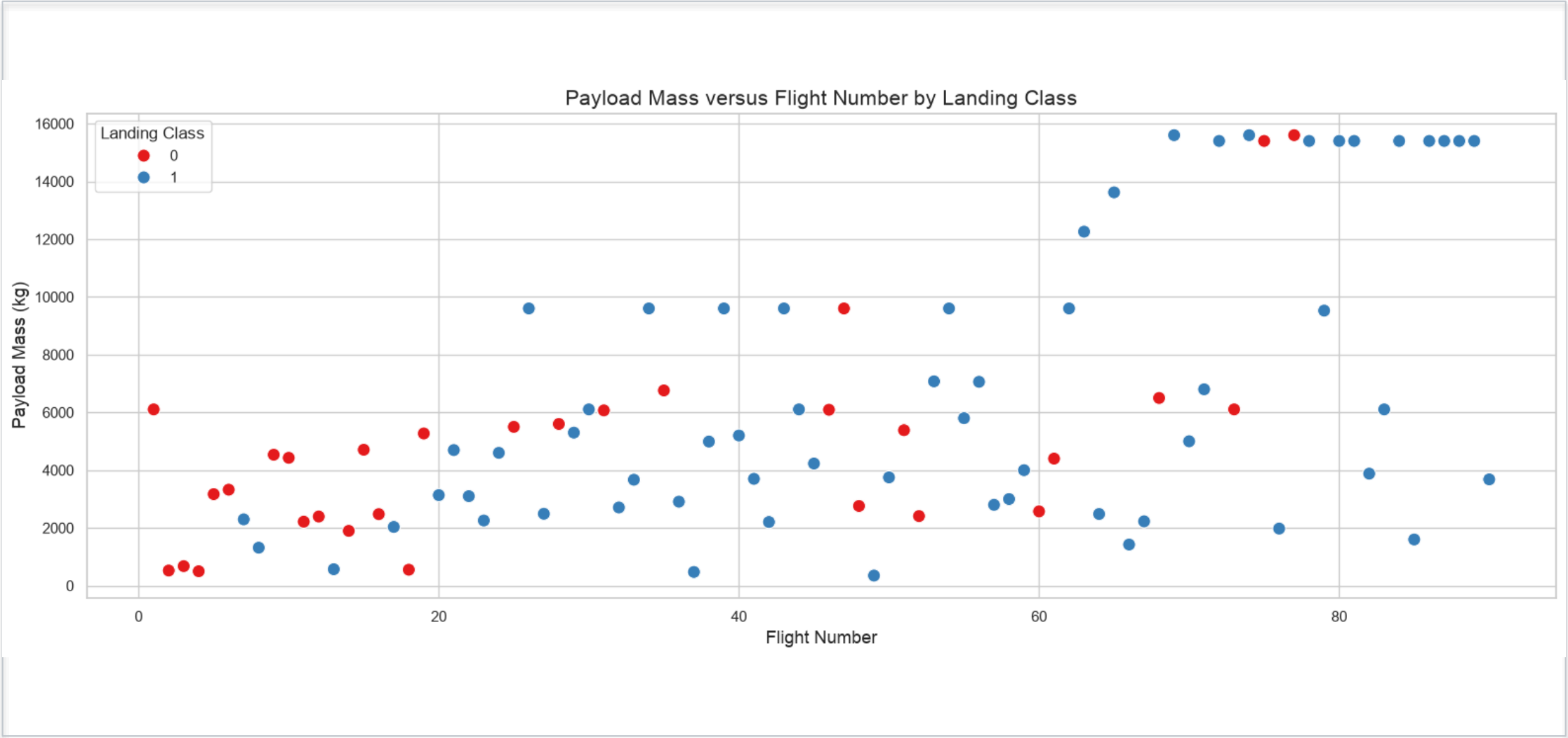
# References and Project Files

Sources, tools and repository evidence

- IBM Applied Data Science Capstone course materials and datasets.
- SpaceX REST API-style launch data and IBM-hosted static SpaceX dataset.
- Wikipedia Falcon 9 and Falcon Heavy launch records used for web scraping.
- Python libraries: pandas, NumPy, BeautifulSoup, SQLite, matplotlib, seaborn, Folium, Plotly Dash and scikit-learn.
- Project repository: <https://github.com/dalalalhabad/IBM-Data-Science-Capstone-SpaceX>

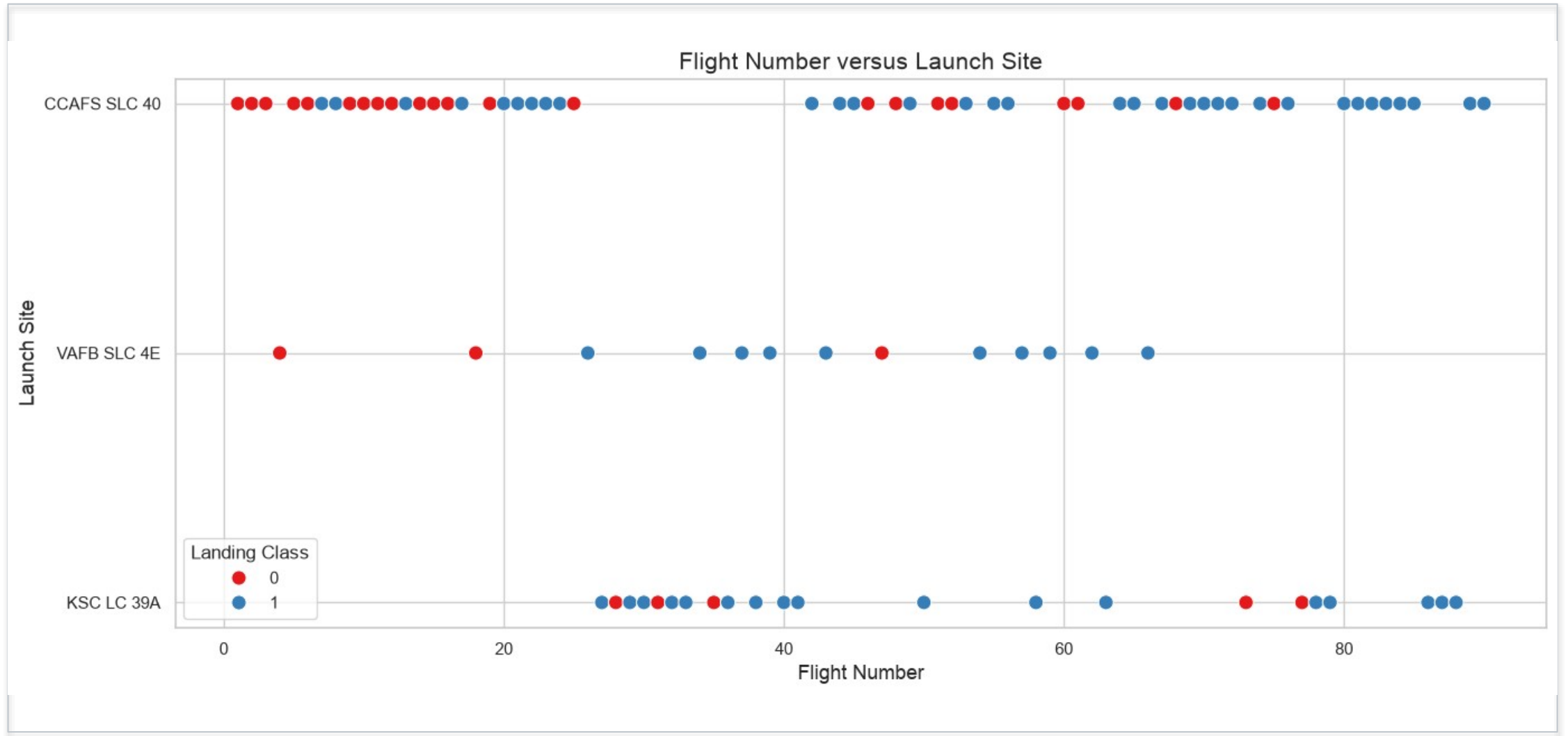
# Full-Size EDA Chart - Payload Mass versus Flight Number

Clean figure extracted from the executed EDA notebook



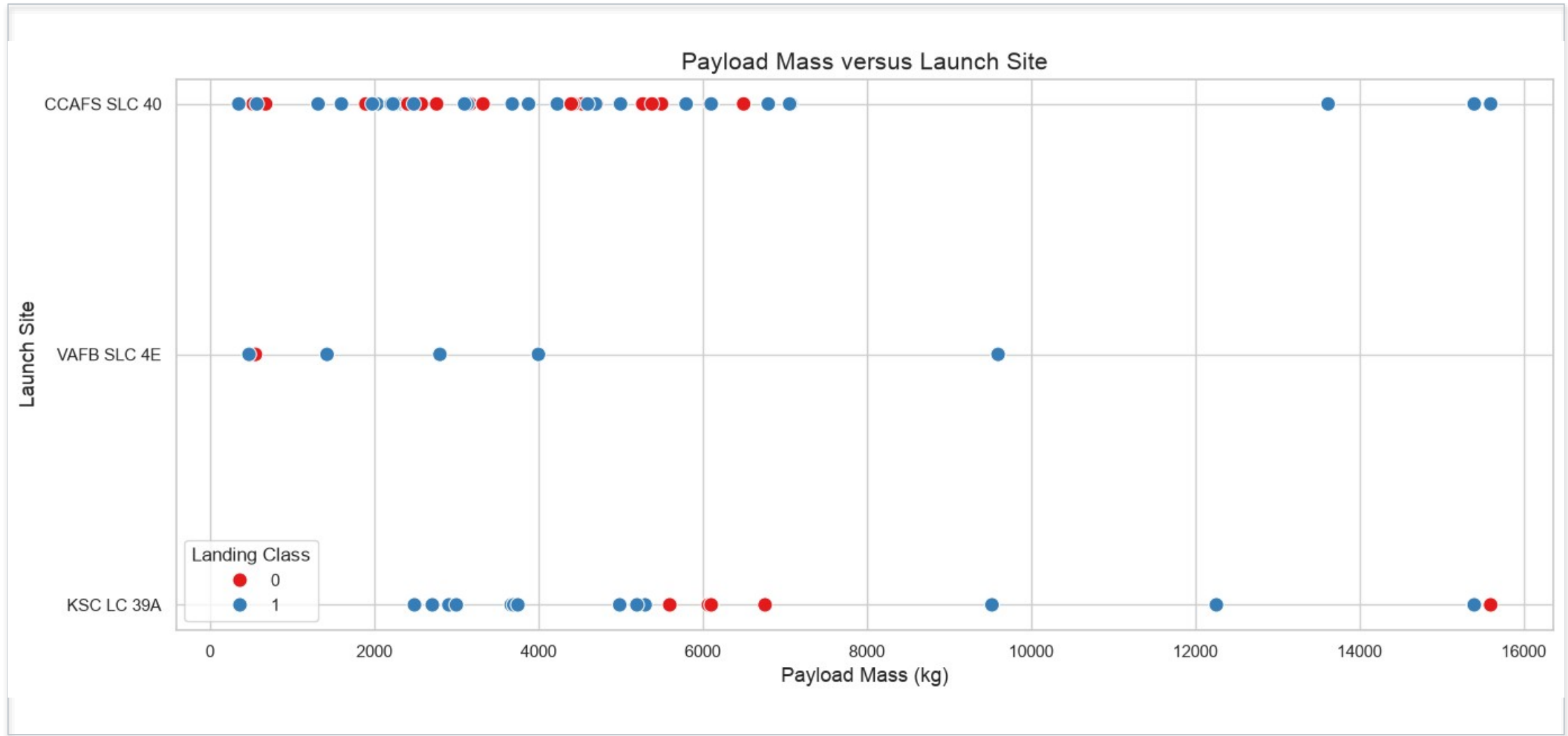
# Full-Size EDA Chart - Flight Number versus Launch Site

Required scatter plot result



# Full-Size EDA Chart - Payload Mass versus Launch Site

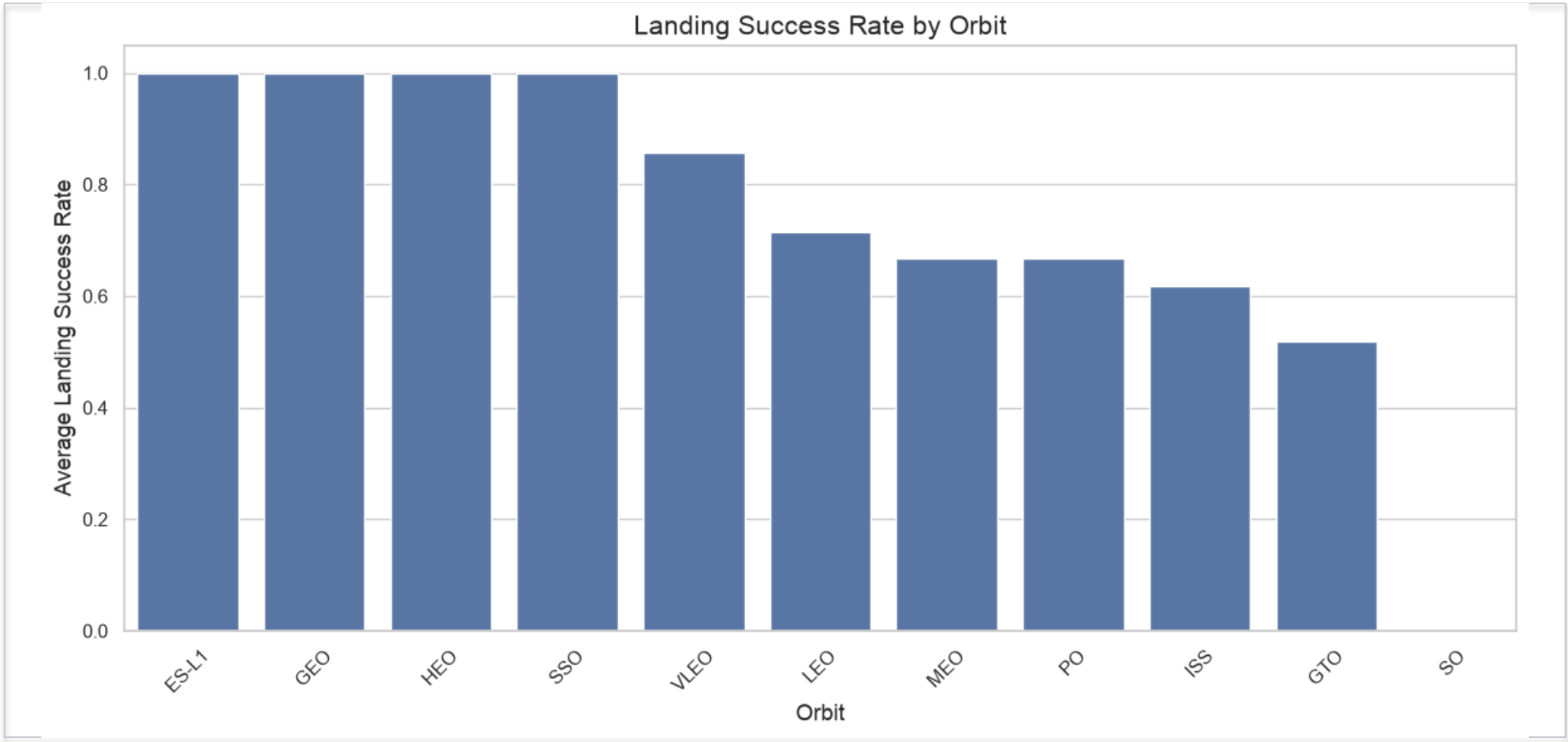
Required scatter plot result





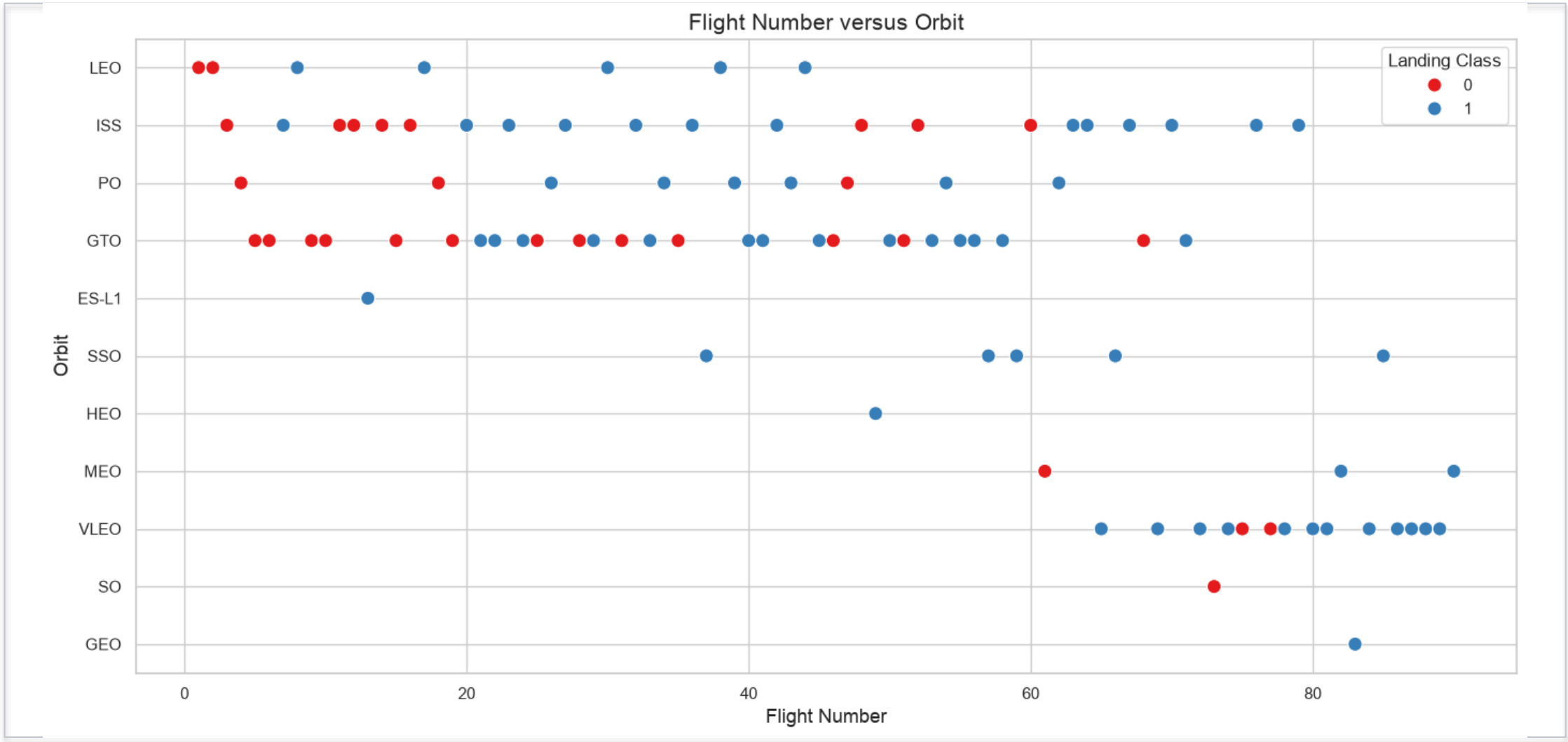
# Full-Size EDA Chart - Success Rate by Orbit

Required bar chart result



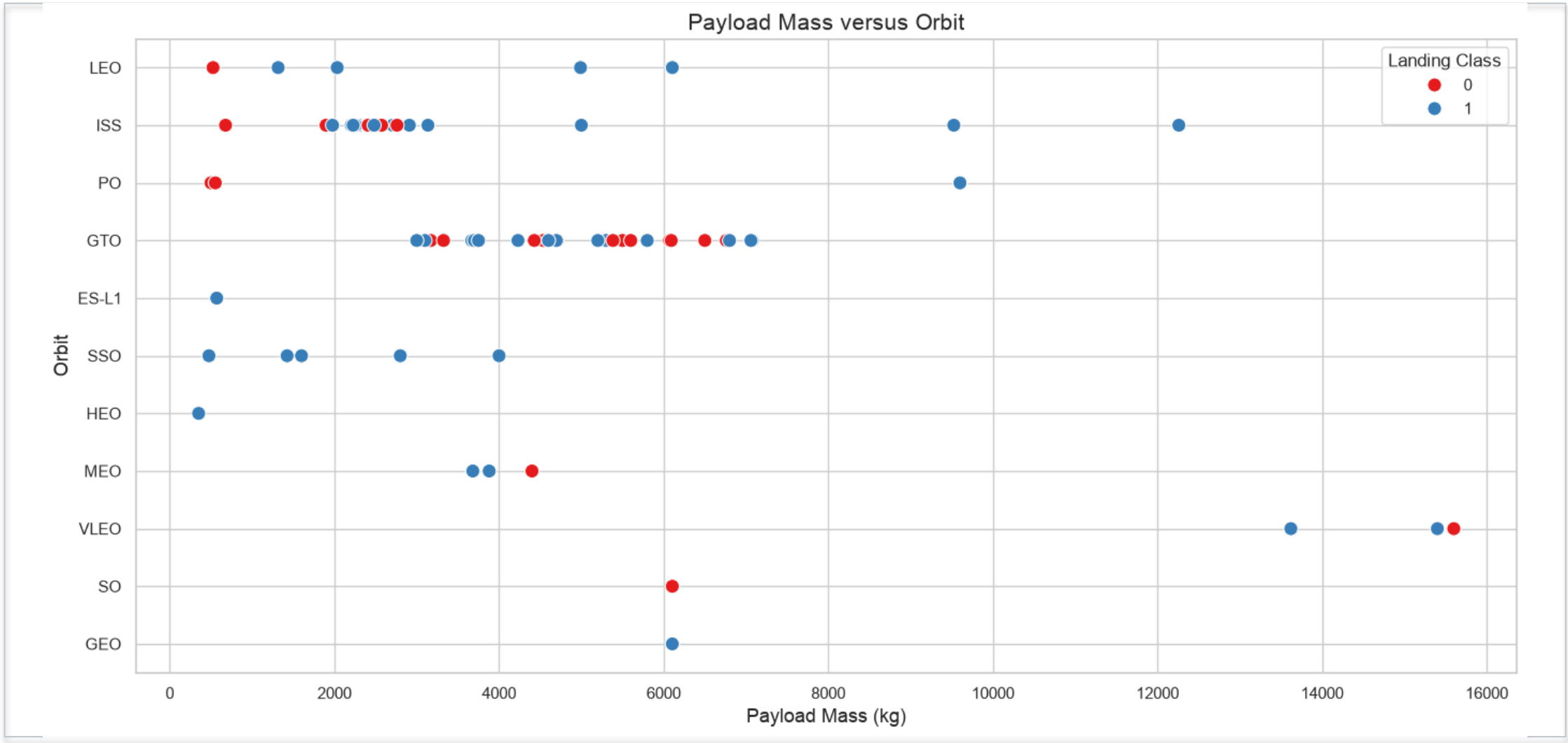
# Full-Size EDA Chart - Flight Number versus Orbit

Required scatter plot result



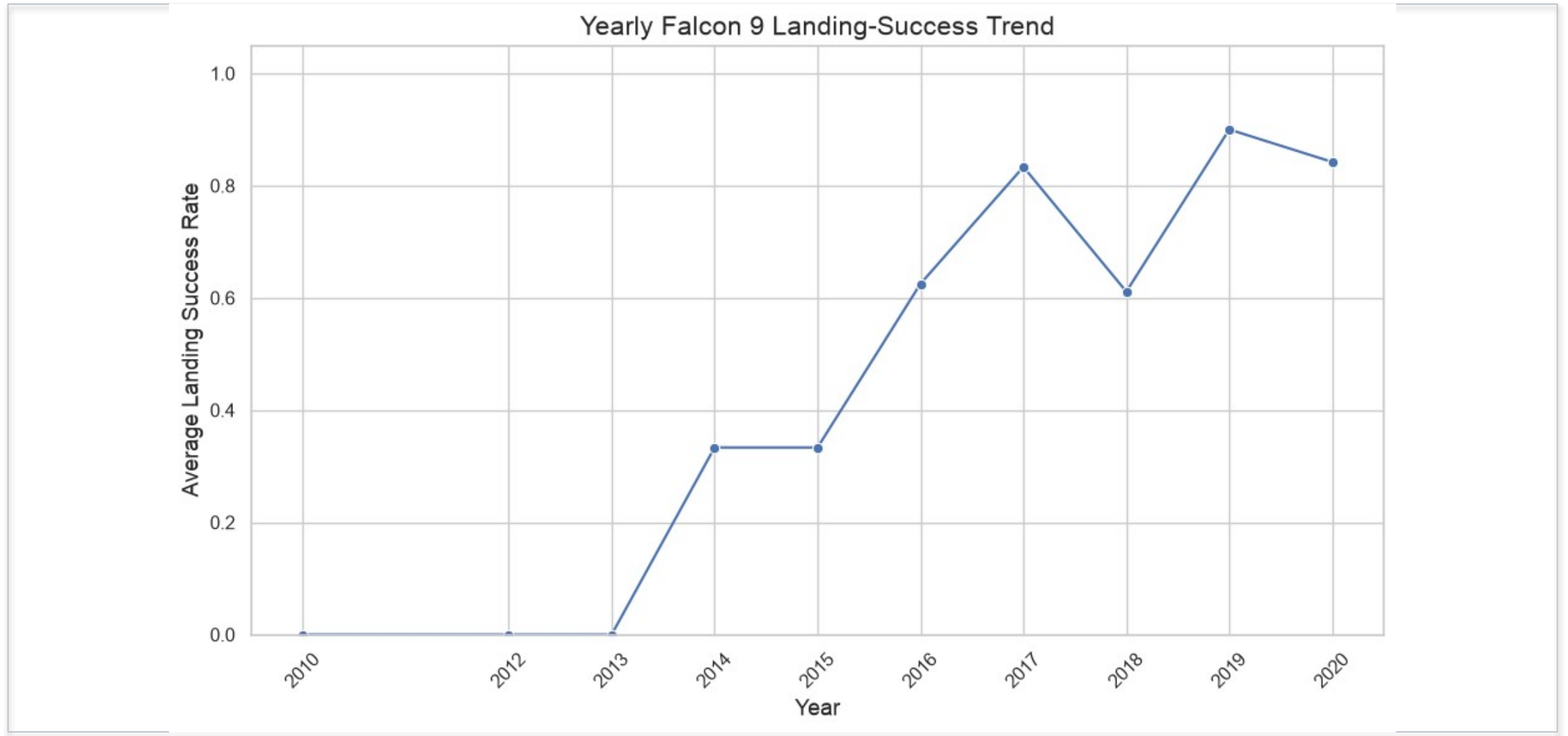
# Full-Size EDA Chart - Payload Mass versus Orbit

Required scatter plot result



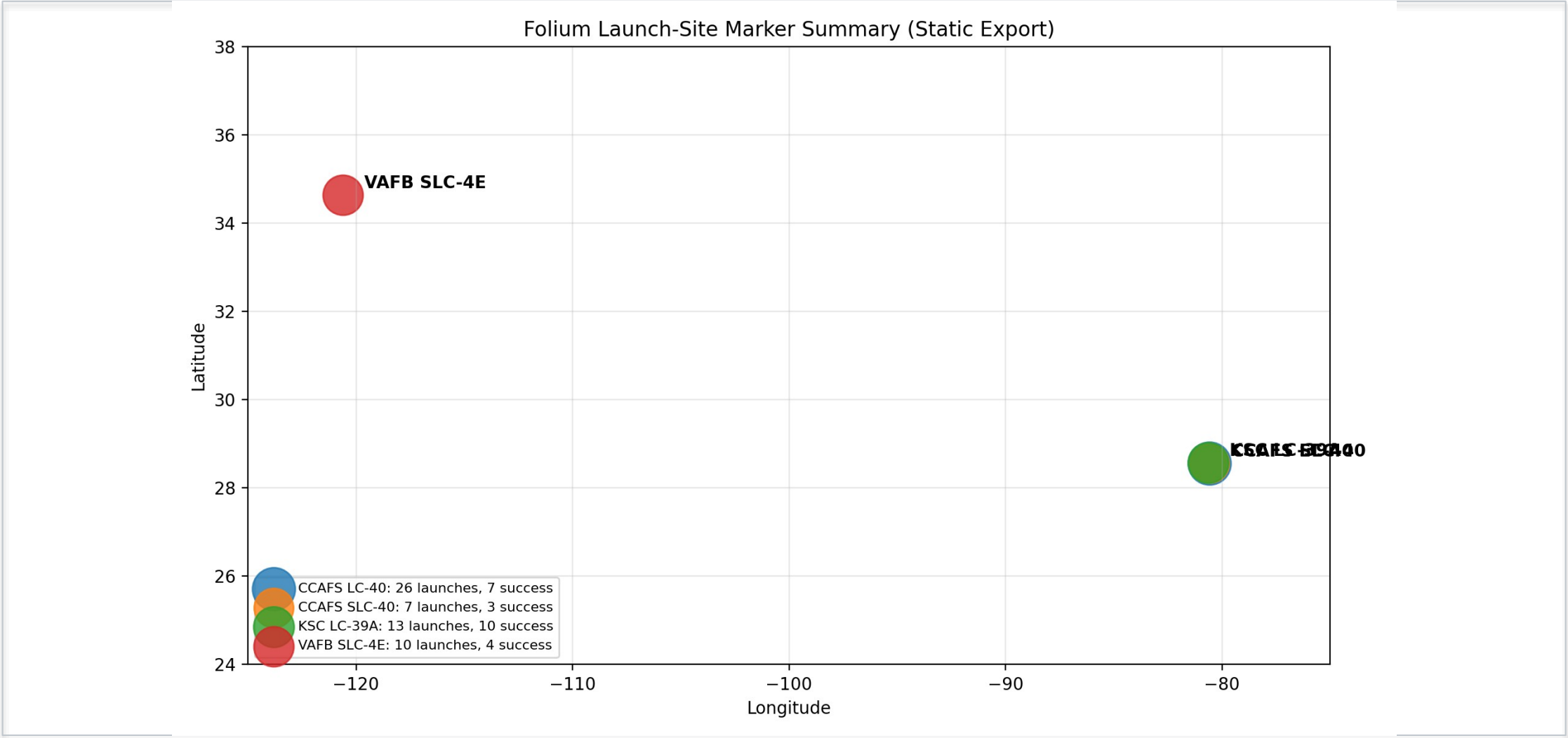
# Full-Size EDA Chart - Yearly Landing Success Trend

Required trend chart result



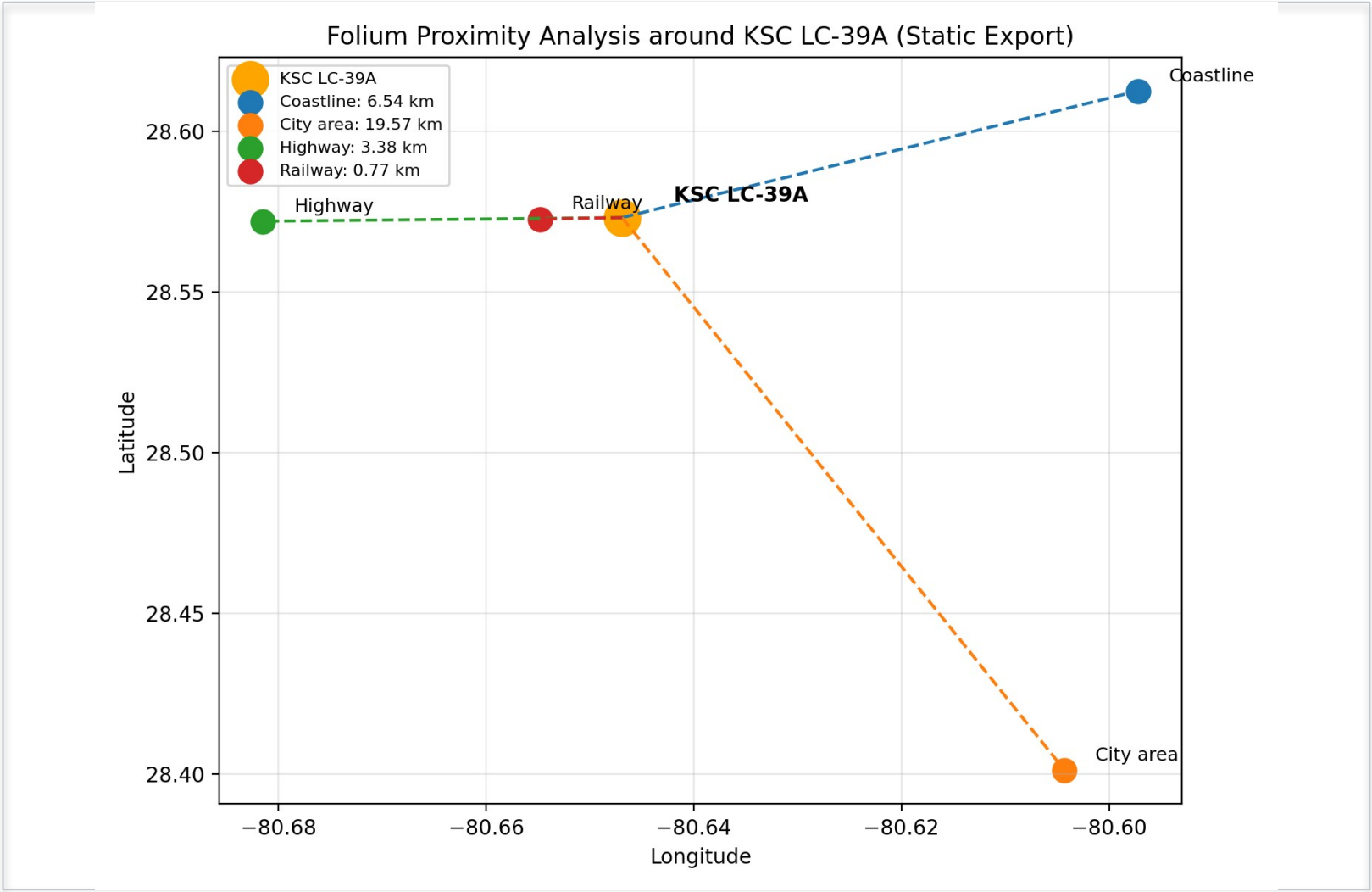
# Full-Size Folium Evidence - Launch Markers

Static export representing launch-site and launch-record markers from the Folium analysis



# Full-Size Folium Evidence - Proximity Analysis

Static export of coastline, city, highway and railway distance analysis around KSC LC-39A



| Feature   | Distance |
|-----------|----------|
| Coastline | 6.54 km  |
| City area | 19.57 km |
| Highway   | 3.38 km  |
| Railway   | 0.77 km  |

# Plotly Dash Source Code Evidence

The completed app contains the dropdown, slider and two callbacks required in the lab

| Required Dash task           | Evidence in completed Python app                                                                                                                   |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Task 1: Launch Site dropdown | <code>dcc.Dropdown(id="site-dropdown", options=[...], value="ALL")</code>                                                                          |
| Task 2: Pie chart callback   | <code>@app.callback(Output("success-pie-chart", "figure"), Input("site-dropdown", "value"))</code>                                                 |
| Task 3: Payload slider       | <code>dcc.RangeSlider(id="payload-slider", min=0, max=10000, step=1000)</code>                                                                     |
| Task 4: Scatter callback     | <code>@app.callback(Output("success-payload-scatter-chart", "figure"), [Input("site-dropdown", "value"), Input("payload-slider", "value")])</code> |

